

Homework 9 Reliability Copilot Report

Wilson Neira

Repo and Starter Code

<https://github.com/wilsonnexus/Building-Scalable-Distributed-Systems/tree/main/HW9/reliability-copilot>

1. Problem, Team, and Overview of Experiments

My project is Reliability Copilot, a small but realistic AWS-based platform for analyzing simulated CI/CD and build/test failures. The system takes in a failure event and returns a failure category, a short recommended action, and a risk score for whether the same kind of failure is likely to happen again. This problem matters because teams lose time when failures are noisy, unclear, or repetitive. A system like this could help engineers triage failures faster and make reliability problems easier to understand. I Wilson Neira am working solo on this project, so I am keeping the project focused and manageable for one person. The main experiments I plan to run are:

1. bottleneck identification
2. scaling behavior under increased load
3. failure injection and recovery

The results I will evaluate include latency, throughput, queue depth, CPU and memory usage, and system behavior during failure conditions. AI will play the role of lightweight classification and recommendation support, while observability will come from logs, metrics, and later CloudWatch dashboards. While LLMs will help with project understanding and coding faster.

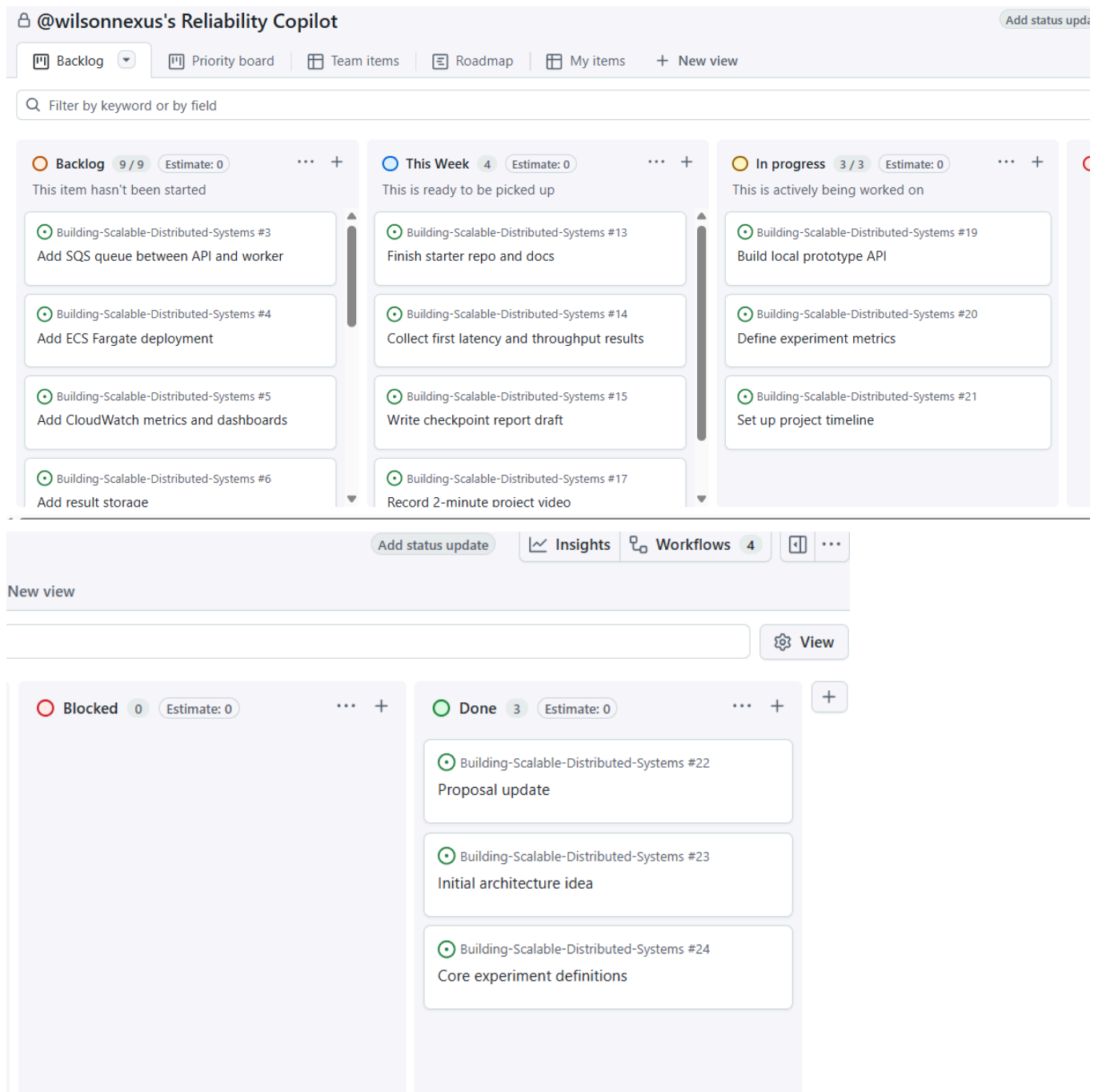
2. Project Plan and Recent Progress

Recent progress includes refining the architecture, defining the three core experiments, creating the starter repository, implementing an initial local API prototype, creating a Locust-based load test, and generating first charts for latency and throughput. Since I am working solo, I am responsible for all parts of the project, including architecture, coding, testing, infrastructure, observability, reporting, poster creation, and presentation.

For March 30 I plan to complete Homework 9's instructions such as a clean repo structure, create starter API and tests, collect first experimental charts, write checkpoint, report draft, and record a 2-minute video. My plan from now until April 19 is to finish the checkpoint materials first, add the queue and worker flow, collect baseline bottleneck results, test scaling behavior, test failure injection and recovery, and finish the final report, poster, and presentation.

I am using AI within the project in two ways. First, the system itself will include lightweight AI or LLM-assisted failure analysis. Second, AI helps me move faster in project development by assisting with drafting, iteration, and code planning. The benefit is faster prototyping and clearer

recommendations. The cost is that I still need to validate outputs carefully, since the real value of the project comes from system design, measurements, and engineering judgment.



3. Objectives

My short-term objective is to build a working prototype that can ingest failure events, analyze them, and produce measurable system behavior under load. My longer-term objective is to turn this into a stronger portfolio project that shows how distributed systems, reliability engineering, and AI can work together in a realistic platform. Observability is also a core objective, not an

extra feature. I want the final project to expose enough logs and metrics that stakeholders can actually understand what is happening when the system slows down or fails. Because the project includes AI components, I also want to control performance, reliability, and cost. That means keeping the AI path lightweight, adding fallback behavior if inference becomes slow, and designing the system so that AI does not become a single point of failure.

4. Related Work

This project connects strongly to distributed systems ideas from the course readings. One important lesson is that distributed systems must be designed with the expectation of failure, rather than assuming that the network and components will always behave well. Another is that latency, availability, and consistency are shaped by real system constraints and tradeoffs.

The project also connects to common distributed systems goals such as scalability, fault tolerance, predictable performance, and recovery from failures. These ideas are directly relevant because Reliability Copilot is not only classifying events, but also trying to remain responsive and observable under changing conditions. These ideas came from these sources:

- My architecture, experiments, and timeline come from your proposal update
- Distributed systems must be designed for failure, latency, availability, and tradeoffs: Distributed systems for fun and profit.
- Distributed systems need fault tolerance, predictable performance, and explicit handling of unreliable communication and failures from: Introduction to Distributed System Design Modified and adapted by James Wilcox from the now-defunct Google Code University tutorial of the same name.

5. Methodology

My methodology is to build the project in layers. First, I will create a basic API that accepts failure events and produces a baseline classification and recommendation. Then I will add queue-based decoupling, a worker path, storage, and observability. After that, I will run experiments that stress the system and measure its behavior.

The three core experiments are:

1. bottleneck identification under increasing load
2. scaling behavior under higher worker capacity or scaling rules
3. failure injection and recovery under conditions such as slow AI, artificial storage delay, or downstream issues

I will use AI in a lightweight way, likely starting with rule-assisted classification or a small inference path before expanding to a fuller LLM-assisted recommendation step. I will support observability with request logs, latency measurements, throughput tracking, queue depth tracking, and infrastructure-level metrics once the AWS version is in place.

The main tradeoffs I am evaluating are: latency vs richer analysis, simplicity vs resilience, stronger processing guarantees vs faster responsiveness, AI usefulness vs AI cost and slowness.

6. Preliminary Results

So far, I have an initial local prototype and a first round of load testing results. What is still left is the more realistic distributed version with queue-based processing, cloud deployment, stronger observability, and the final experiment set. The worst-case workload for this project is a sudden burst of many failure events at once, especially if the AI path or storage path is slow. The base-case workload is a steady stream of modest event traffic with low contention.

```
Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems
$ curl http://localhost:8080/health
{"status":"ok"}

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)
$ curl -X POST http://localhost:8080/analyze \
  -H "Content-Type: application/json" \
  -d '{"pipeline_id":"p1","log_text":"job failed due to timeout while calling dependency service","source":"jenkins",
"timestamp":"2026-03-28T12:00:00Z"}'
{"failure_category":"timeout","recommendation":"Check service latency, retry settings, and downstream availability.",
"risk_score":0.82,"processing_mode":"rule-based-baseline"}
```

locust -f .\tests\locustfile.py --host http://localhost:8080 --headless -u 20 -r 5 -t 1m --csv .\results\initial

HW9 > reliability-copilot > results > initial_exceptions.csv > data

1 Count, Message, Traceback, Nodes

HW9 > reliability-copilot > results > initial_failures.csv > data

1 Method, Name, Error, Occurrences

HW9 > reliability-copilot > results > initial_stats_history.csv > data

1 Timestamp, User Count, Type, Name, Requests/s, Failures/s, 50%, 66%, 75%, 80%
100
101 1774747877, 20, , Aggregated, 65.300000, 0.000000, 2, 3, 3, 3, 4, 4, 6, 12, 57, 94,
102
103 1774747878, 20, , Aggregated, 65.200000, 0.000000, 2, 3, 3, 3, 4, 4, 6, 11, 57, 94,

HW9 > reliability-copilot > results > initial_stats.csv > data

1 Type, Name, Request Count, Failure Count, Median Response Time, Average Response
2
3 POST, /analyze, 3835, 0, 2, 2.6976930656903844, 0.9294000919908285, 94.489600043743
4
5 , Aggregated, 3835, 0, 2, 2.6976930656903844, 0.9294000919908285, 94.48960004374385

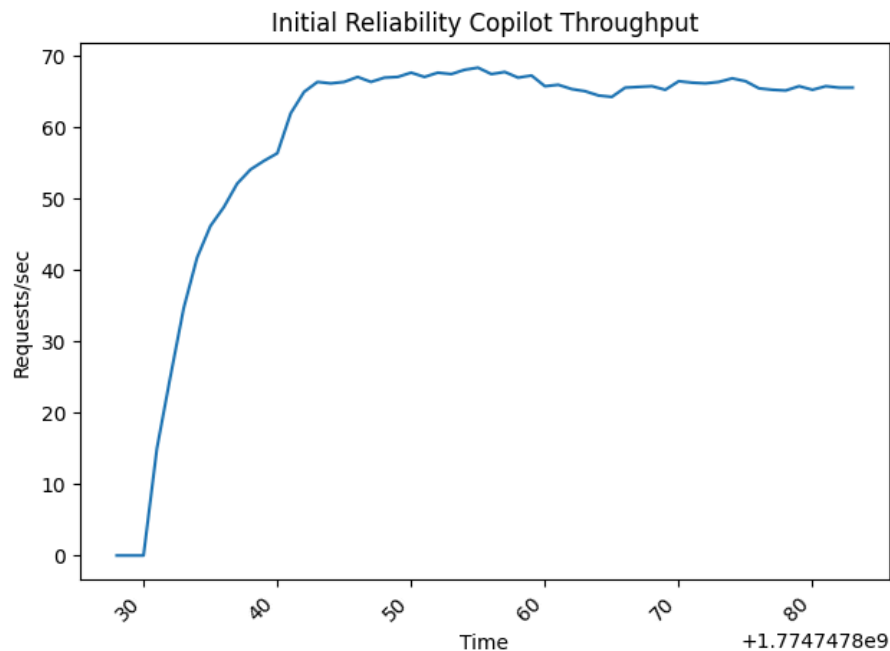
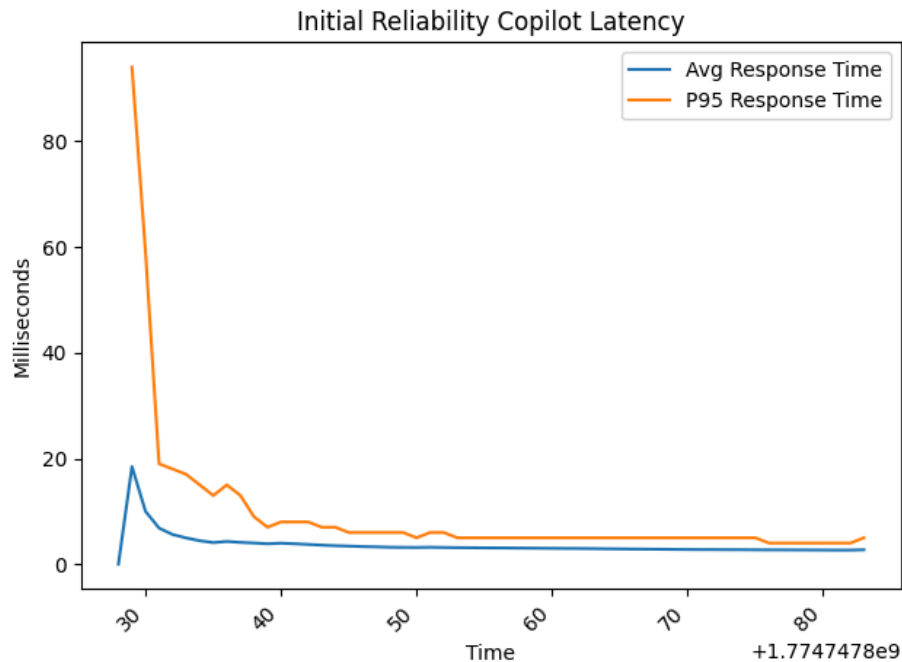
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW9\reliability-copilot> python .\tests\plot_results.py

Columns found:

['Timestamp', 'User Count', 'Type', 'Name', 'Requests/s', 'Failures/s', '50%', '66%', '75%', '80%', '90%', '95%', '98%', '99%', '99.9%', '99.99%', '100%', 'Total Request Count', 'Total Failure Count', 'Total Median Response Time', 'Total Average Response Time', 'Total Min Response Time', 'Total Max Response Time', 'Total Average Content Size']

Saved throughput_chart.png using column: Requests/s

Saved latency_chart.png using columns: Total Average Response Time, 95%



7. Impact

I think people would care about this project because it connects several things that are useful in real engineering work: platform reliability, distributed systems design, observability, and practical AI assistance. This could also be useful to other students if they want to help test the API or compare failure patterns. Even as a course project, it has value as a portfolio piece because it is not just another CRUD app or isolated AI demo. It is a system with measurable tradeoffs and stakeholder value.